

PYTHON

Complete Handwritten Notes

Beginner → Intermediate → Advanced

Learn Python • Practice Python • Build Projects

Notebook Goal — A complete revision guide from Python basics to OOP, files, exceptions, modules, tools, NumPy/Pandas and an AI-engineering path.

Made by Gitansh

1. Python Fundamentals

Python

- High-level, readable, general-purpose language.
- Used for AI/ML, data science, automation, web, APIs, testing and scripting.
- Python uses indentation to define blocks.

```
print("Hello, World!")
```

Variables & Types

- Variables are names bound to objects; no type declaration is required.
- Core types: int, float, complex, bool, str, list, tuple, set, dict, None.

```
name = "Gitansh"  
age = 14  
pi = 3.14  
active = True  
nums = [1,2,3]  
point = (10,20)  
unique = {1,2,3}  
user = {"name":"Gitansh"}
```

Comments

- Use # for comments. Triple-quoted strings are commonly used as docstrings.

```
# comment  
def add(a,b):  
    """Return the sum."""  
    return a+b
```

2. Input, Output & Operators

Input / Print

- `input()` returns `str`, so convert numeric input.
- `print()` supports `sep` and `end`.

```
name=input("Name: ")
age=int(input("Age: "))
print("Hello",name)
print("A","B","C",sep="-",end="!")
```

Operators

- **Arithmetic:** `+` `-` `*` `/` `//` `%` `**`.
- **Comparison:** `==` `!=` `>` `<` `>=` `<=`.
- **Logical:** `and` or `not`.
- **Assignment:** `=` `+=` `-=` `*=` `/=` `//=` `%=` `**=`.
- **Membership:** `in`, `not in`. **Identity:** `is`, `is not`.

```
a,b=10,3
print(a+b,a/b,a//b,a%b,a**b)
print(a>b and b>0)
print("py" in "python")
```

3. Strings

String Basics

- *Strings are immutable Unicode sequences.*
- *Index from 0; negative indexes start at -1. Slicing is start:stop:step.*

```
s="Python"
print(s[0],s[-1])
print(s[1:4])
print(s[::-1])
```

Methods & Formatting

- *Important methods: lower, upper, title, strip, replace, split, join, startswith, endswith, find, count.*
- *f-strings are the preferred readable formatting style.*

```
text="  hello python  "
print(text.strip().title())
print("-".join(["2026","10","04"]))
name="Gitansh"
print(f"Hi {name}")
```

4. Conditions

if / elif / else

- *Conditions select a block. Indentation is required.*

```
marks=82
if marks>=90:
    grade="A+"
elif marks>=75:
    grade="A"
elif marks>=60:
    grade="B"
else:
    grade="C"
print(grade)
```

Nested & Ternary

- *Nested if means an if inside another block. Conditional expressions are useful for short choices.*

```
age=16
if age>=13:
    if age<18:
        print("Teen")
status="Pass" if marks>=40 else "Fail"
```

5. Loops

for

- Iterates over strings, lists, tuples, sets, dictionaries and other iterables.

```
for i in range(1,6):  
    print(i)  
for ch in "Python":  
    print(ch)
```

while

- Repeats while a condition is true. Change the state so the loop can terminate.

```
n=1  
while n<=5:  
    print(n)  
    n+=1
```

break / continue / pass

- *break* exits a loop; *continue* skips an iteration; *pass* does nothing and acts as a placeholder.

```
for n in range(1,11):  
    if n==5: break  
    if n==2: continue  
    print(n)
```

Nested Loops

- Useful for tables, grids, matrices and patterns.

```
for i in range(3):  
    for j in range(3):  
        print("*",end=" ")  
    print()
```

6. Lists

Basics

- Lists are ordered and mutable. They can contain mixed types.

```
nums=[10,20,30]
nums[1]=25
nums.append(40)
print(nums)
```

Methods

- *append, extend, insert, remove, pop, sort, reverse, clear* are important.

```
nums=[3,1,2]
nums.append(4)
nums.extend([5,6])
nums.insert(0,9)
nums.remove(1)
last=nums.pop()
nums.sort()
```

Slicing & Comprehension

- *Slicing uses start:stop:step. Comprehensions create collections compactly.*

```
nums=[1,2,3,4,5]
print(nums[1:4])
squares=[n*n for n in nums]
evens=[n for n in nums if n%2==0]
```

7. Tuples, Sets & Dictionaries

Tuples

- Ordered and immutable. Useful for fixed data and unpacking.

```
point=(10,20)
x,y=point
print(x,y)
```

Sets

- Store unique values. Operators: | union, & intersection, – difference, ^ symmetric difference.

```
a={1,2,3}
b={3,4,5}
print(a|b)
print(a&b)
print(a-b)
```

Dictionaries

- Store key-value pairs. Common methods: get, keys, values, items, update, pop.

```
student={"name":"Gitansh","age":14}
print(student.get("marks",0))
student["age"]=15
for k,v in student.items():
    print(k,v)
```

Comprehensions

- List, set and dictionary comprehensions are useful for transformations.

```
squares={n:n*n for n in range(1,6)}
unique={x for x in [1,1,2,3]}
```

8. Functions

def / return

- Functions group reusable logic. Parameters receive values; return sends a result.

```
def add(a,b):  
    return a+b  
print(add(5,3))
```

Arguments

- Positional arguments depend on order; keyword arguments use names; defaults provide fallback values; *args collects extra positional arguments; **kwargs collects extra keyword arguments.

```
def greet(name,message="Hello"):  
    print(message,name)  
greet("Gitansh")  
def total(*numbers):  
    return sum(numbers)  
print(total(1,2,3))
```

Scope & lambda

- Local variables belong to functions. Avoid unnecessary global state. lambda creates a small anonymous function.

```
double=lambda x:x*2  
print(double(5))  
names=["Bob","Alex","Chris"]  
print(sorted(names,key=lambda x:len(x)))
```

9. Modules & Packages

Imports

- A module is a .py file containing reusable code. A package groups modules.

```
import math
print(math.sqrt(25))
from random import randint
print(randint(1,10))
```

Standard Library

- Useful modules: *math*, *random*, *datetime*, *pathlib*, *os*, *json*, *re*, *statistics*, *collections*.

```
from pathlib import Path
p=Path("data.txt")
print(p.exists())
import json
print(json.dumps({"name":"Gitansh"}))
```

10. File Handling

Text Files

- Use with `open(...)` so files close automatically. Modes: `r`, `w`, `a`, `x`, `b`, `+`.

```
with open("notes.txt","w",encoding="utf-8") as f:
    f.write("Learn Python")
with open("notes.txt","r",encoding="utf-8") as f:
    print(f.read())
```

CSV

- The `csv` module handles tabular comma-separated data. `DictReader` gives named columns.

```
import csv
with open("students.csv",newline="",encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["name"])
```

JSON

- `JSON` is common in APIs/configuration. `dump/load` work with files; `dumps/loads` work with strings.

```
import json
data={"name":"Gitansh","skills":["Python","NumPy"]}
with open("data.json","w",encoding="utf-8") as f:
    json.dump(data,f,indent=4)
```

11. Exceptions

try / except

- Catch specific exceptions when possible. else runs after success; finally always runs.

```
try:
    a=int(input("A: "))
    b=int(input("B: "))
    print(a/b)
except ValueError:
    print("Invalid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("Success")
finally:
    print("Finished")
```

raise & Custom Exception

- raise deliberately creates an exception. Custom exceptions inherit from Exception.

```
def set_age(age):
    if age<0:
        raise ValueError("Age cannot be negative")
    return age
class InvalidMarksError(Exception):
    pass
```

12. OOP

Class & Object

- A class defines structure/behavior; an object is an instance. `__init__` initializes; `self` refers to the instance.

```
class Student:
    def __init__(self,name,marks):
        self.name=name
        self.marks=marks
    def result(self):
        return "Pass" if self.marks>=40 else "Fail"
s=Student("Gitansh",85)
print(s.name,s.result())
```

Four Pillars

- Encapsulation keeps data and behavior together. Abstraction hides unnecessary detail. Inheritance reuses a base class. Polymorphism allows a common interface with different implementations.

```
class Animal:
    def speak(self):
        print("Some sound")
class Dog(Animal):
    def speak(self):
        print("Bark")
for a in [Animal(),Dog()]:
    a.speak()
```

Static / Class Methods

- `@staticmethod` receives neither `self` nor `cls` automatically. `@classmethod` receives `cls`.

```
class MathTools:
    @staticmethod
    def square(x):
        return x*x
print(MathTools.square(5))
```

13. Iterators & Generators

Iterators

- Iterators implement the iteration protocol. for loops use `iter()` and `next()` internally.

```
nums=iter([10,20,30])
print(next(nums))
print(next(nums))
```

Generators

- Generators use `yield` and produce values lazily, saving memory for large sequences.

```
def count_up_to(n):
    i=1
    while i<=n:
        yield i
        i+=1
for x in count_up_to(5):
    print(x)
```

14. Decorators

Decorator

- A decorator wraps a function to add behavior without changing its core code.

```
def log_call(func):
    def wrapper(*args,**kwargs):
        print("Calling...")
        result=func(*args,**kwargs)
        print("Done")
        return result
    return wrapper
@log_call
def greet():
    print("Hello")
greet()
```

15. Regular Expressions

re

- *Regex matches text patterns. Useful symbols: . ^ \$ * + ? [] \d \w.*

```
import re
text="My score is 95"
m=re.search(r"\d+",text)
if m:
    print(m.group())
```

16. Type Hints & Dataclasses

Type Hints

- *Type hints document intended types and help tools; Python normally does not enforce them at runtime.*

```
def add(a:int,b:int)->int:  
    return a+b  
name:str="Gitansh"
```

Dataclass

- *dataclasses reduce boilerplate for data-focused classes.*

```
from dataclasses import dataclass  
@dataclass  
class User:  
    name:str  
    age:int  
u=User("Gitansh",14)  
print(u)
```

17. Virtual Environments & pip

venv

- *Virtual environments isolate dependencies per project.*

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install requests
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
```

Good Practice

- *Prefer `python -m pip` so `pip` belongs to the intended Python interpreter. Keep dependencies documented.*

18. Python Libraries

AI / Data

• NumPy: numerical arrays. Pandas: tabular data. Matplotlib: plots. scikit-learn: classical ML. PyTorch: deep learning. TensorFlow/Keras: deep learning.

Automation / Apps

• requests: HTTP. Flask/FastAPI: APIs. BeautifulSoup: HTML parsing. OpenPyXL: Excel. Pillow: images. pytest: testing.

19. Python for AI Engineering

Roadmap

• 1 Python → 2 functions/OOP/files/exceptions → 3 NumPy → 4 Pandas → 5 visualization → 6 linear algebra/probability/statistics → 7 scikit-learn → 8 PyTorch → 9 APIs/Git/testing/environments → 10 deployment/projects.

NumPy

• Learn arrays, shape, dtype, indexing, slicing, broadcasting, reshape, concatenate, split, random and linear algebra.

```
import numpy as np
a=np.array([[1,2],[3,4]])
print(a.shape)
print(a[0,1])
print(a*2)
print(a.T)
```

Pandas

• Learn read_csv, head, info, describe, loc, iloc, filtering, groupby, merge, missing values and export.

```
import pandas as pd
df=pd.read_csv("data.csv")
print(df.head())
print(df[df["marks"]>=80])
```

20. Debugging & Errors

Common Errors

• *SyntaxError: invalid syntax. NameError: unknown name. TypeError: incompatible types. ValueError: invalid value. IndexError: bad index. KeyError: missing dictionary key. FileNotFoundError: missing file. ZeroDivisionError: division by zero.*

Debugging Checklist

• *Read the traceback from the bottom. Check line numbers. Print or inspect values. Reduce to a small reproducible case. Test edge cases.*

21. Clean Code

Best Practices

- Use clear names and 4 spaces. Keep functions focused. Prefer readability. Handle expected errors. Write docstrings. Use virtual environments. Keep API keys out of source code. Use Git and meaningful commits.

Naming

- *snake_case*: variables/functions. *PascalCase*: classes. *UPPER_CASE*: constants.

```
MAX_RETRIES=3
student_name="Gitansh"
def calculate_score(marks):
    return marks
class StudentRecord:
    pass
```

22. Mini Projects

Beginner

- Calculator, number guessing game, quiz, password generator, unit converter, expense tracker, contact book.

Intermediate

- Weather app, CSV analyzer, PDF merger, file organizer, stock analyzer, Flask notes app, web scraper.

AI / Portfolio

- Spam classifier, image classifier, face-recognition prototype, recommender, sentiment analyzer, chatbot, document Q&A, AI study planner.

Project Formula

- Problem → input/data → processing → output → error handling → tests → README → GitHub → demo.

23. Quick Revision

Must-Know

- *print(), input(), if/elif/else, for, while, def, return, try/except/finally, class, import, with open, comprehensions.*

```
def function_name(parameter):  
    return result  
  
with open("file.txt","r",encoding="utf-8") as f:  
    data=f.read()
```

Built-ins

- *len, type, range, enumerate, zip, sum, min, max, sorted, reversed, abs, round, any, all, map, filter, isinstance.*

Mindset

- *Break large problems into small functions. Code regularly. Read errors carefully. Build projects, not only tutorials.*

24. Final Python Cheat Sheet

Data Structures

```
list_=[1,2,3]
tuple_=(1,2,3)
set_={1,2,3}
dict_={"a":1,"b":2}
```

Control Flow

```
if condition:
    ...
for item in items:
    ...
while condition:
    ...
```

Function

```
def function_name(parameter):
    return result
```

Exception

```
try:
    risky_code()
except ValueError as e:
    print(e)
```

OOP

```
class Person:
    def __init__(self,name):
        self.name=name
```

Remember — Consistency beats speed. Code regularly, build projects, revise mistakes and keep improving your portfolio.

Made by Gitansh